

NAND2TETRIS

Project 1 & Project 2 — HDL Implementations

Name: Nimarta Kaur Thind

Roll no: 2501420013

Course: B.Tech CSE (Data Science)

Section: A

Semester: 3

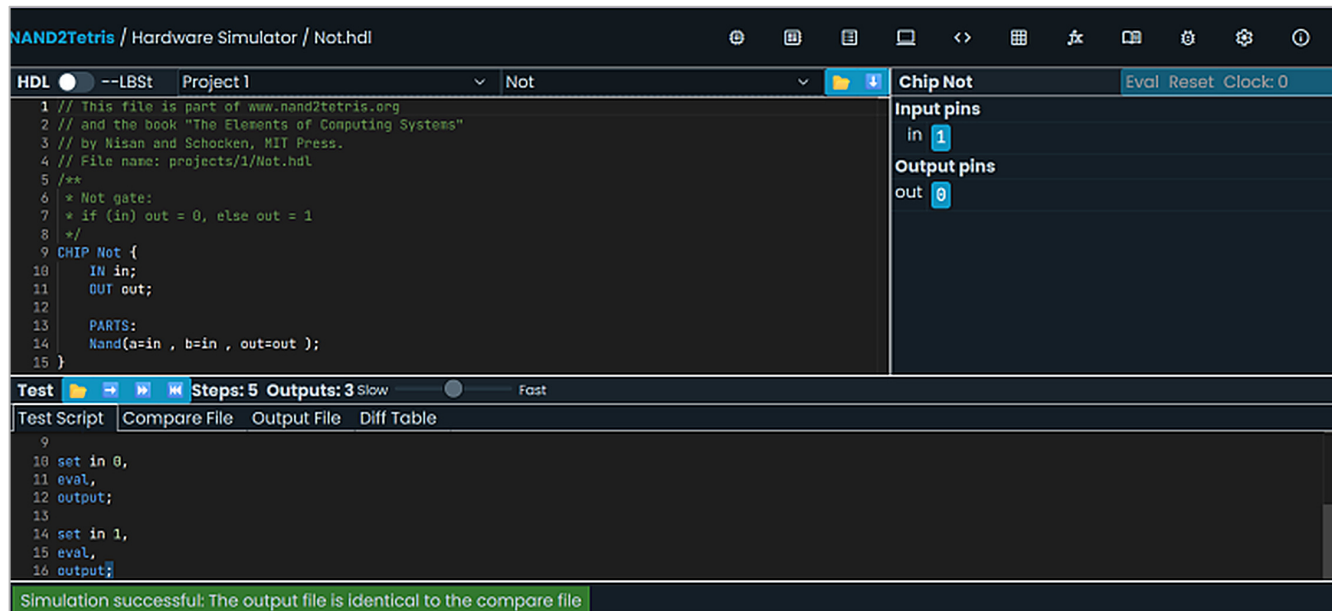
PROJECT 1: BOOLEAN LOGIC

Implementation of elementary and composite logic gates, built starting from the primitive Nand gate, as specified in Chapter 1 of *The Elements of Computing Systems*.

NOT GATE

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Not.hdl
/**
 * Not gate:
 * if (in) out = 0, else out = 1
 */
CHIP Not {
    IN in;
    OUT out;

    PARTS:
        Nand(a=in , b=in , out=out );
}
```



Simulator output: Not Gate

AND GATE

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And.hdl
/**
 * And gate:
 * if (a and b) out = 1, else out = 0
 */
CHIP And {
    IN a, b;
    OUT out;

    PARTS:
        Nand(a=a , b=b , out=c );
        Nand(a=c , b=c , out=out );
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top panel shows the HDL code for the 'And' chip, which is a 2-input, 1-output chip. The code uses two Nand gates to implement the AND function. The right panel shows the chip's configuration: input pins 'a' and 'b' are both set to 1, the output pin 'out' is 1, and the internal pin 'c' is 0. The bottom panel shows the test script, which sets 'b' to 0, evaluates the output, then sets 'a' to 1, sets 'b' to 1, evaluates the output, and finally outputs the result. A green status bar at the bottom indicates 'Simulation successful: The output file is identical to the compare file'.

NAND2Tetris / Hardware Simulator / And.hdl

HDL ☐ --LBST Project 1 And Chip And Eval Reset Clock: 0

```
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/And.hdl
5 /**
6  * And gate:
7  * if (a and b) out = 1, else out = 0
8  */
9 CHIP And {
10     IN a, b;
11     OUT out;
12
13     PARTS:
14         Nand(a=a , b=b , out=c );
15         Nand(a=c , b=c , out=out );
16 }
```

Test ☐ Steps: 7 Outputs: 5 Slow Fast

Test Script Compare File Output File Diff Table

```
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: And Gate

OR GATE

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or.hdl
/**
 * Or gate:
 * if (a or b) out = 1, else out = 0
 */
CHIP Or {
    IN a, b;
    OUT out;

    PARTS:
    Nand(a=a, b=a, out=x1);
    Nand(a=b , b=b , out=x2 );
    Nand(a=x1 , b=x2 , out= out);
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top toolbar includes icons for file operations, simulation control, and settings. The main window is divided into three sections:

- HDL Editor:** Shows the Verilog code for the Or gate, with line numbers 1 through 15. The code implements an OR gate using two NAND gates.
- Chip Or:** A panel on the right showing the internal state of the chip. It includes input pins 'a' and 'b', both set to 1; an output pin 'out' set to 1; and internal pins 'x1' and 'x2', both set to 0.
- Test Script:** A panel at the bottom showing a sequence of test steps: 21 set b 0, 22 eval, 23 output, 24, 25 set a 1, 26 set b 1, 27 eval, 28 output. A green status bar at the bottom indicates "Simulation successful: The output file is identical to the compare file".

Simulator output: Or Gate

XOR GATE

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Xor.hdl
/**
 * Exclusive-or gate:
 * if ((a and Not(b)) or (Not(a) and b)) out = 1, else out = 0
 */
CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
        Nand(a=a , b=b , out=x1 );
        Nand(a=a , b=x1 , out=x2 );
        Nand(a=b , b=x1 , out=x3);
        Nand(a=x2 , b=x3 , out=out);
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top window shows the HDL code for the Xor gate. The bottom window shows the simulation results, including the Test Script and the output file.

Chip Xor Eval Reset Clock: 0

Input pins

- a: 1
- b: 1

Output pins

- out: 0

Internal pins

- x1: 0
- x2: 1
- x3: 1

Test Steps: 7 Outputs: 5 Slow Fast

Test Script Compare File Output File Diff Table

```
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: Xor Gate

MUX (MULTIPLEXOR)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux.hdl
/**
 * Multiplexor:
 * if (sel = 0) out = a, else out = b
 */
CHIP Mux {
    IN a, b, sel;
    OUT out;

    PARTS:
    Not(in=sel, out=x);
    And(a=x, b=a, out=w1);
    And(a=sel, b=b, out=w2);
    Or(a=w1, b=w2, out=out);
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the file path "NAND2Tetris / Hardware Simulator / Mux.hdl". The main window is divided into three sections:

- HDL Editor:** Contains the Verilog code for the Mux chip, identical to the code shown in the first block.
- Chip Mux:** A panel on the right showing the internal state of the chip. It includes:
 - Input pins:** a (1), b (1), sel (1).
 - Output pins:** out (1).
 - Internal pins:** x (0), w1 (0).
- Test Panel:** At the bottom, it shows a test script with the following commands:

```
41 set b 1,
42 set sel 0,
43 eval,
44 output;
45
46 set sel 1,
47 eval,
48 output;
```

A green status bar at the bottom indicates: "Simulation successful: The output file is identical to the compare file".

Simulator output: Mux

DMUX (DEMULTIPLEXOR)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux.hdl
/**
 * Demultiplexor:
 * [a, b] = [in, 0] if sel = 0
 *          [0, in] if sel = 1
 */
CHIP DMux {
    IN in, sel;
    OUT a, b;

    PARTS:
    Not(in= sel, out=x1 );
    And(a=in , b=x1 , out=a );
    And(a=in, b=sel , out=b );
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top window shows the HDL code for the DMux chip, which is a demultiplexor. The code defines inputs 'in' and 'sel', and outputs 'a' and 'b'. It uses a NOT gate to create 'x1' (the inverse of 'sel'), and two AND gates to route 'in' to either 'a' or 'b' based on the value of 'sel'. The bottom window shows the simulation results. The 'Test' tab is active, displaying a sequence of operations: setting 'in' to 1, setting 'sel' to 0, evaluating, and outputting. The output shows 'a' as 0 and 'b' as 1. A green status bar at the bottom indicates 'Simulation successful: The output file is identical to the compare file'.

NAND2Tetris / Hardware Simulator / DMux.hdl

HDL --LBSt Project1 DMux

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/DMux.hdl
5 /**
6  * Demultiplexor:
7  * [a, b] = [in, 0] if sel = 0
8  *          [0, in] if sel = 1
9  */
10 CHIP DMux {
11     IN in, sel;
12     OUT a, b;
13
14     PARTS:
15     Not(in= sel, out=x1 );
16 }
```

Chip DMux Eval Reset Clock: 0

Input pins

in 1

sel 1

Output pins

a 0

b 1

Internal pins

x1 0

Test Steps: 7 Outputs: 5 Slow Fast

Test Script Compare File Output File Diff Table

```
19 set in 1,
20 set sel 0,
21 eval,
22 output;
23
24 set sel 1,
25 eval,
26 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: DMux

NOT16 (16-BIT NOT GATE)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/01/Not16.hdl
/**
 * 16-bit Not gate:
 * for i = 0, ..., 15:
 * out[i] = Not(a[i])
 */
CHIP Not16 {
    IN in[16];
    OUT out[16];

    PARTS:
    Not(in=in[0], out=out[0]);
    Not(in=in[1], out=out[1]);
    Not(in=in[2], out=out[2]);
    Not(in=in[3], out=out[3]);
    Not(in=in[4], out=out[4]);
    Not(in=in[5], out=out[5]);
    Not(in=in[6], out=out[6]);
    Not(in=in[7], out=out[7]);
    Not(in=in[8], out=out[8]);
    Not(in=in[9], out=out[9]);
    Not(in=in[10], out=out[10]);
    Not(in=in[11], out=out[11]);
    Not(in=in[12], out=out[12]);
    Not(in=in[13], out=out[13]);
    Not(in=in[14], out=out[14]);
    Not(in=in[15], out=out[15]);
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris / Hardware Simulator / Not16.hdl". The main window is divided into two panes. The left pane shows the HDL code for the Not16 chip, which is a 16-bit NOT gate. The right pane shows the chip's configuration, including input and output pins. The input pins are set to 0001001000110100, and the output pins are set to 1110110111001011. The bottom status bar indicates that the simulation is successful and the output file is identical to the compare file.

Chip Not16 Eval Reset Clock: 0

Input pins
in 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec

Output pins
out 1 1 1 0 1 1 0 1 1 1 0 0 1 0 1 1 dec

Test Steps: 8 Outputs: 6 Slow Fast

Test Script Compare File Output File Diff Table

```
21
22 set in %B0011110011000011,
23 eval,
24 output;
25
26 set in %B0001001000110100,
27 eval,
28 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: Not16

AND16 (16-BIT AND GATE)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And16.hdl
/**
 * 16-bit And gate:
 * for i = 0, ..., 15:
 * out[i] = a[i] And b[i]
 */
CHIP And16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    And(a=a[0], b=b[0], out=out[0]);
    And(a=a[1], b=b[1], out=out[1]);
    And(a=a[2], b=b[2], out=out[2]);
    And(a=a[3], b=b[3], out=out[3]);
    And(a=a[4], b=b[4], out=out[4]);
    And(a=a[5], b=b[5], out=out[5]);
    And(a=a[6], b=b[6], out=out[6]);
    And(a=a[7], b=b[7], out=out[7]);
    And(a=a[8], b=b[8], out=out[8]);
    And(a=a[9], b=b[9], out=out[9]);
    And(a=a[10], b=b[10], out=out[10]);
    And(a=a[11], b=b[11], out=out[11]);
    And(a=a[12], b=b[12], out=out[12]);
    And(a=a[13], b=b[13], out=out[13]);
    And(a=a[14], b=b[14], out=out[14]);
    And(a=a[15], b=b[15], out=out[15]);
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris / Hardware Simulator / And16.hdl". The main window is divided into three sections:

- HDL Editor:** Contains the Verilog code for the And16 chip, which implements a 16-bit AND gate by performing 16 individual AND operations on the input bits.
- Chip And16:** Shows the chip's configuration with input pins 'a' and 'b', and output pins 'out'. The input 'a' is set to 00010001000110100 and input 'b' is set to 10011000001110110. The output 'out' is shown as 00010000000110100.
- Test Script:** Contains a test script that sets inputs 'a' and 'b' to specific 16-bit values, evaluates the chip, and outputs the result. The script is as follows:

```
31 set b %0000011111110000,
32 eval,
33 output;
34
35 set a %0001001000110100,
36 set b %1001100000111010,
37 eval,
38 output;
```

At the bottom, a green status bar indicates: "Simulation successful: The output file is identical to the compare file".

Simulator output: And16

OR16 (16-BIT OR GATE)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or16.hdl
/**
 * 16-bit Or gate:
 * for i = 0, ..., 15:
 * out[i] = a[i] Or b[i]
 */
CHIP Or16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    Or(a=a[0], b=b[0], out=out[0]);
    Or(a=a[1], b=b[1], out=out[1]);
    Or(a=a[2], b=b[2], out=out[2]);
    Or(a=a[3], b=b[3], out=out[3]);
    Or(a=a[4], b=b[4], out=out[4]);
    Or(a=a[5], b=b[5], out=out[5]);
    Or(a=a[6], b=b[6], out=out[6]);
    Or(a=a[7], b=b[7], out=out[7]);
    Or(a=a[8], b=b[8], out=out[8]);
    Or(a=a[9], b=b[9], out=out[9]);
    Or(a=a[10], b=b[10], out=out[10]);
    Or(a=a[11], b=b[11], out=out[11]);
    Or(a=a[12], b=b[12], out=out[12]);
    Or(a=a[13], b=b[13], out=out[13]);
    Or(a=a[14], b=b[14], out=out[14]);
    Or(a=a[15], b=b[15], out=out[15]);
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top toolbar includes icons for file operations, simulation control, and settings. The main window is divided into three sections:

- HDL Editor:** Shows the Verilog code for the Or16 chip, including comments and the main chip definition with input/output ports and internal OR gate components.
- Chip Or16:** A panel on the right showing the current state of the chip. It includes input pins 'a' and 'b', and the output pin 'out'. Each pin is represented by a 16-bit binary value and a 'dec' button for decimal conversion.
- Test Console:** A bottom section showing the test script and the results of the simulation. It includes a 'Test' button, a progress bar, and a table for test results.

The test script shows the following steps:

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/Or16.hdl
5 /**
6  * 16-bit Or gate:
7  * for i = 0, ..., 15:
8  * out[i] = a[i] Or b[i]
9  */
10 CHIP Or16 {
11     IN a[16], b[16];
12     OUT out[16];
13
14     PARTS:
15     Or(a=a[0], b=b[0], out=out[0]);
16
17     Or(a=a[1], b=b[1], out=out[1]);
18     Or(a=a[2], b=b[2], out=out[2]);
19     Or(a=a[3], b=b[3], out=out[3]);
20     Or(a=a[4], b=b[4], out=out[4]);
21     Or(a=a[5], b=b[5], out=out[5]);
22     Or(a=a[6], b=b[6], out=out[6]);
23     Or(a=a[7], b=b[7], out=out[7]);
24     Or(a=a[8], b=b[8], out=out[8]);
25     Or(a=a[9], b=b[9], out=out[9]);
26     Or(a=a[10], b=b[10], out=out[10]);
27     Or(a=a[11], b=b[11], out=out[11]);
28     Or(a=a[12], b=b[12], out=out[12]);
29     Or(a=a[13], b=b[13], out=out[13]);
30     Or(a=a[14], b=b[14], out=out[14]);
31     Or(a=a[15], b=b[15], out=out[15]);
32 }
33
34 Test
35 set b %0000011111110000,
36 eval,
37 output;
38
39 set a %00001001000110100,
40 set b %01001100000110110,
41 eval,
42 output;
```

The simulation results show that the output is identical to the compare file, indicating a successful simulation.

Simulator output: Or16

MUX16 (16-BIT MULTIPLEXOR)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux16.hdl
/**
 * 16-bit multiplexor:
 * for i = 0, ..., 15:
 * if (sel = 0) out[i] = a[i], else out[i] = b[i]
 */
CHIP Mux16 {
    IN a[16], b[16], sel;
    OUT out[16];
    PARTS:
    Mux(a=a[0], b=b[0], sel=sel, out=out[0]);
    Mux(a=a[1], b=b[1], sel=sel, out=out[1]);
    Mux(a=a[2], b=b[2], sel=sel, out=out[2]);
    Mux(a=a[3], b=b[3], sel=sel, out=out[3]);
    Mux(a=a[4], b=b[4], sel=sel, out=out[4]);
    Mux(a=a[5], b=b[5], sel=sel, out=out[5]);
    Mux(a=a[6], b=b[6], sel=sel, out=out[6]);
    Mux(a=a[7], b=b[7], sel=sel, out=out[7]);
    Mux(a=a[8], b=b[8], sel=sel, out=out[8]);
    Mux(a=a[9], b=b[9], sel=sel, out=out[9]);
    Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
    Mux(a=a[11], b=b[11], sel=sel, out=out[11]);
    Mux(a=a[12], b=b[12], sel=sel, out=out[12]);
    Mux(a=a[13], b=b[13], sel=sel, out=out[13]);
    Mux(a=a[14], b=b[14], sel=sel, out=out[14]);
    Mux(a=a[15], b=b[15], sel=sel, out=out[15]);
}
```

NAND2Tetris / Hardware Simulator / Mux16.hdl

HDL --LBSt Project 1 Mux16 Chip Mux16 Eval Reset Clock: 0

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/Mux16.hdl
5 /**
6  * 16-bit multiplexor:
7  * for i = 0, ..., 15:
8  * if (sel = 0) out[i] = a[i], else out[i] = b[i]
9  */
10 CHIP Mux16 {
11     IN a[16], b[16], sel;
12     OUT out[16];
13
14     PARTS:
15     Mux(a=a[0], b=b[0], sel=sel, out=out[0]);
```

Input pins

a 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 dec

b 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec

sel 1

Output pins

out 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec

Test Steps: 11 Outputs: 9 Slow Fast

Test Script Compare File Output File Diff Table

```
41 set b %80101010101010101,
42 set sel 0,
43 eval,
44 output;
45
46 set sel 1,
47 eval,
48 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: Mux16

OR8Way (8-WAY OR GATE)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or8Way.hdl
/**
 * 8-way Or gate:
 * out = in[0] Or in[1] Or ... Or in[7]
 */
CHIP Or8Way {
    IN in[8];
    OUT out;

    PARTS:
    Or(a=in[0], b=in[1], out=x1);
    Or(a=in[2], b=in[3], out=x2);
    Or(a=in[4], b=in[5], out=x3);
    Or(a=in[6], b=in[7], out=x4);

    Or(a=x1, b=x2, out=x5);
    Or(a=x3, b=x4, out=x6);

    Or(a=x5, b=x6, out=out);
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top bar displays the file path "NAND2Tetris / Hardware Simulator / Or8Way.hdl". The main editor shows the HDL code for the Or8Way chip. The right panel shows the chip's configuration, including input pins (in: 0 0 1 0 0 1 1 0), output pins (out: 1), and internal pins (x1: 1, x2: 1, x3: 1, x4: 0). The bottom panel shows the test script and the simulation results.

Test Script:

```
21
22 set in %00000001,
23 eval,
24 output;
25
26 set in %000100110,
27 eval,
28 output;
```

Simulation Results:

Simulation successful: The output file is identical to the compare file

Simulator output: Or8Way

MUX4WAY16 (4-WAY 16-BIT MULTIPLEXOR)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux4Way16.hdl
/**
 * 4-way 16-bit multiplexor:
 * out = a if sel = 00
 *      b if sel = 01
 *      c if sel = 10
 *      d if sel = 11
 */
CHIP Mux4Way16 {
    IN a[16], b[16], c[16], d[16], sel[2];
    OUT out[16];

    PARTS:
        Mux16(a=a, b=b, sel=sel[0], out=ab);
        Mux16(a=c, b=d, sel=sel[0], out=cd);
        Mux16(a=ab, b=cd, sel=sel[1], out=out);
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris / Hardware Simulator / Mux4Way16.hdl". The main window is divided into several sections:

- HDL Editor:** Contains the Verilog code for the Mux4Way16 chip, including comments, a 4-way 16-bit multiplexor logic, and a test script.
- Chip Mux4Way16:** A panel showing the internal structure of the chip, including input pins (a, b, c, d, sel) and output pins (out).
- Test Script:** A section showing the test script used to verify the chip's functionality.
- Simulation Status:** A green bar at the bottom indicating "Simulation successful: The output file is identical to the compare file".

The test script in the HDL Editor shows the following steps:

```
41
42 set sel 2,
43 eval,
44 output;
45
46 set sel 3,
47 eval,
48 output;
```

The chip configuration panel shows the following values:

- Input pins: a = 0001001000110100, b = 1001100000111011, c = 1010101010101010, d = 0101010101010101.
- sel = 11.
- Output pins: out = 0101010101010101.

Simulator output: Mux4Way16

MUX8WAY16 (8-WAY 16-BIT MULTIPLEXOR)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux8Way16.hdl
/**
 * 8-way 16-bit multiplexor:
 * out = a if sel = 000
 *      b if sel = 001
 *      c if sel = 010
 *      d if sel = 011
 *      e if sel = 100
 *      f if sel = 101
 *      g if sel = 110
 *      h if sel = 111
 */
CHIP Mux8Way16 {
    IN a[16], b[16], c[16], d[16],
        e[16], f[16], g[16], h[16],
        sel[3];
    OUT out[16];

    PARTS:
    Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1], out=abcd);
    Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1], out=efgh);
    Mux16(a=abcd, b=efgh, sel=sel[2], out=out);
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top bar displays the project name "NAND2Tetris / Hardware Simulator / Mux8Way16.hdl". The main window is divided into three sections:

- HDL Editor:** Displays the Verilog code for the Mux8Way16 chip, including comments and the main logic block.
- Chip Mux8Way16:** Shows the input pins (a, b, c, d, e, f, g, h) and their corresponding 16-bit values. Each input has a "dec" button for decimal conversion.
- Test Console:** Shows the test script and the simulation results. The test script includes commands like "set sel 6", "eval", "output", "set sel 7", "eval", and "output". The simulation results show the output of the chip for each test case.

The bottom status bar indicates "Simulation successful: The output file is identical to the compare file".

Simulator output: Mux8Way16

DMUX4WAY (4-WAY DEMULTIPLEXOR)

```
// This file is part of www.nand2tetrtris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux4Way.hdl
/**
 * 4-way demultiplexor:
 * [a, b, c, d] = [in, 0, 0, 0] if sel = 00
 *                  [0, in, 0, 0] if sel = 01
 *                  [0, 0, in, 0] if sel = 10
 *                  [0, 0, 0, in] if sel = 11
 */
CHIP DMux4Way {
    IN in, sel[2];
    OUT a, b, c, d;

    PARTS:
    DMux(in=in, sel=sel[1], a=a, b=b);
    DMux(in=a, sel=sel[0], a=a, b=b);
    DMux(in=c, sel=sel[0], a=c, b=d);
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top bar displays the file path: NAND2Tetris / Hardware Simulator / DMux4Way.hdl. Below the bar, the HDL code for the DMux4Way chip is shown in a text editor. The code defines a 4-way demultiplexor that takes an input 'in' and a 2-bit selector 'sel' and produces four outputs 'a', 'b', 'c', and 'd'. The selector 'sel' is currently set to '11' (decimal 3). The output pins are shown on the right: 'a' is 0, 'b' is 0, 'c' is 0, and 'd' is 1. The internal pins section is also visible. At the bottom, the Test panel shows the test script and the simulation results. The test script sets 'sel' to '10' and '11', evaluates the chip, and outputs the results. The simulation results show that the output is identical to the compare file.

Test **Steps: 11** **Outputs: 9** **Slow** **Fast**

Test Script Compare File Output File Diff Table

```
35
36 set sel %B10,
37 eval,
38 output;
39
40 set sel %B11,
41 eval,
42 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: DMux4Way

DMUX8WAY (8-WAY DEMULTIPLEXOR)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux8Way.hdl
/**
 * 8-way demultiplexor:
 * [a, b, c, d, e, f, g, h] = [in, 0, 0, 0, 0, 0, 0, 0] if sel = 000
 *                               [0, in, 0, 0, 0, 0, 0, 0] if sel = 001
 *                               [0, 0, in, 0, 0, 0, 0, 0] if sel = 010
 *                               [0, 0, 0, in, 0, 0, 0, 0] if sel = 011
 *                               [0, 0, 0, 0, in, 0, 0, 0] if sel = 100
 *                               [0, 0, 0, 0, 0, in, 0, 0] if sel = 101
 *                               [0, 0, 0, 0, 0, 0, in, 0] if sel = 110
 *                               [0, 0, 0, 0, 0, 0, 0, in] if sel = 111
 */
CHIP DMux8Way {
    IN in, sel[3];
    OUT a, b, c, d, e, f, g, h;

    PARTS:
    DMux(in=in, sel=sel[2], a=abcd, b=efgh);

    DMux4Way(in=abcd, sel=sel[0..1], a=a, b=b, c=c, d=d);
    DMux4Way(in=efgh, sel=sel[0..1], a=e, b=f, c=g, d=h);
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top bar displays the project name "NAND2Tetris / Hardware Simulator / DMux8Way.hdl". The main window is divided into two panes. The left pane shows the HDL code for the DMux8Way chip, which is an 8-way demultiplexor. The right pane shows the chip configuration, including input pins (in, sel) and output pins (a, b, c, d, e, f, g, h). The simulation is running, and the output pins are shown as 0. The bottom status bar indicates "Simulation successful: The output file is identical to the compare file".

Chip DMux8Way Eval Reset Clock: 0

Input pins

- in: 1
- sel: 1 1 1 (dec)

Output pins

- a: 0
- b: 0
- c: 0
- d: 0
- e: 0

Test Steps: 19 Outputs: 17 Slow Fast

Test Script Compare File Output File Diff Table

```
67
68 set sel %B110,
69 eval,
70 output;
71
72 set sel %B111,
73 eval,
74 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: DMux8Way

PROJECT 2: BOOLEAN ARITHMETIC

Implementation of the chips that make up the Hack computer's arithmetic logic unit (ALU), built on top of the Project 1 gate library, as specified in Chapter 2 of *The Elements of Computing Systems*.

HALFADDER

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/HalfAdder.hdl
/**
 * Computes the sum of two bits.
 */
CHIP HalfAdder {
    IN a, b;      // 1-bit inputs
    OUT sum,      // Right bit of a + b
        carry;   // Left bit of a + b

    PARTS:
        Xor(a=a , b=b , out=sum );
        And(a=a , b=b , out=carry );
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top bar displays the file path: **NAND2Tetris / Hardware Simulator / HalfAdder.hdl**. Below this, the HDL code for the HalfAdder chip is shown in a text editor. To the right of the code, the 'Chip HalfAdder' configuration panel is visible, showing input pins 'a' and 'b' both set to 1, and output pins 'sum' set to 0 and 'carry' set to 1. Below the code editor, the 'Test' panel shows a script with 28 steps, with the first 7 steps completed. The script includes commands like 'set b 0', 'eval', 'output', 'set a 1', 'set b 1', 'eval', and 'output'. At the bottom, a green status bar indicates: 'Simulation successful: The output file is identical to the compare file'.

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/2/HalfAdder.hdl
5 /**
6  * Computes the sum of two bits.
7  */
8 CHIP HalfAdder {
9     IN a, b;      // 1-bit inputs
10    OUT sum,      // Right bit of a + b
11        carry;   // Left bit of a + b
12
13    PARTS:
14        Xor(a=a , b=b , out=sum );
15        And(a=a , b=b , out=carry );
16
17 }
18
19
20
21
22
23
24
25
26
27
28
```

Input pins
a 1
b 1
Output pins
sum 0
carry 1

Test Script Compare File Output File Diff Table

```
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: HalfAdder

FULLADDER

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/FullAdder.hdl
/**
 * Computes the sum of three bits.
 */
CHIP FullAdder {
    IN a, b, c; // 1-bit inputs
    OUT sum,      // Right bit of a + b + c
        carry;   // Left bit of a + b + c

    PARTS:
    HalfAdder(a=a, b=b, sum=sum1, carry=carry1);
    HalfAdder(a=sum1, b=c, sum=sum, carry=carry2);
    Or(a=carry1, b=carry2, out=carry);
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top window shows the HDL code for the FullAdder chip. The bottom window shows the test script and the simulation results.

Test Script:

```
39 set b 1,
40 set c 0,
41 eval,
42 output;
43
44 set c 1,
45 eval,
46 output;
```

Simulation Results:

Simulation successful: The output file is identical to the compare file

Chip FullAdder Pin Values:

Pin	Value
a	1
b	1
c	1
sum	1
carry	1
sum1	0

Simulator output: FullAdder

ADD16 (16-BIT ADDER)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/Add16.hdl
/**
 * 16-bit adder: Adds two 16-bit two's complement values.
 * The most significant carry bit is ignored.
 */
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    FullAdder(a=a[0], b=b[0], c=false, sum=out[0], carry=c1);
    FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
    FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
    FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
    FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
    FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
    FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
    FullAdder(a=a[7], b=b[7], c=c7, sum=out[7], carry=c8);
    FullAdder(a=a[8], b=b[8], c=c8, sum=out[8], carry=c9);
    FullAdder(a=a[9], b=b[9], c=c9, sum=out[9], carry=c10);
    FullAdder(a=a[10], b=b[10], c=c10, sum=out[10], carry=c11);
    FullAdder(a=a[11], b=b[11], c=c11, sum=out[11], carry=c12);
    FullAdder(a=a[12], b=b[12], c=c12, sum=out[12], carry=c13);
    FullAdder(a=a[13], b=b[13], c=c13, sum=out[13], carry=c14);
    FullAdder(a=a[14], b=b[14], c=c14, sum=out[14], carry=c15);
    FullAdder(a=a[15], b=b[15], c=c15, sum=out[15], carry=overflow);
}
```

NAND2Tetris / Hardware Simulator / Add16.hdl

HDL --LBSt Project 2 Add16 Chip Add16 Eval Reset Clock: 0

1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/2/Add16.hdl
5 /**
6 * 16-bit adder: Adds two 16-bit two's complement values.
7 * The most significant carry bit is ignored.
8 */
9 CHIP Add16 {
10 IN a[16], b[16];
11 OUT out[16];
12
13 PARTS:
14 FullAdder(a=a[0], b=b[0], c=false, sum=out[0], carry=c1);
15 FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);

Input pins
a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0
b 1 0 0 1 1 0 0 0 0 0 1 1 1 0 1 1 0

Output pins
out 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0

Internal pins
c1 0
c2 0
c3 0

Test Steps: 9 Outputs: 7 Slow Fast

Test Script Compare File Output File Diff Table

```
31 set b %00000111111110000,  
32 eval,  
33 output;  
34  
35 set a %0001001000110100,  
36 set b %1001100001110110,  
37 eval,  
38 output;
```

Simulation successful: The output file is identical to the compare file

Simulator output: Add16

ALU (ARITHMETIC LOGIC UNIT)

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/ALU.hdl
/**
 * ALU (Arithmetic Logic Unit):
 * Computes out = one of the following functions:
 *
 *      0, 1, -1,
 *      x, y, !x, !y, -x, -y,
 *      x + 1, y + 1, x - 1, y - 1,
 *      x + y, x - y, y - x,
 *      x & y, x | y
 * on the 16-bit inputs x, y,
 * according to the input bits zx, nx, zy, ny, f, no.
 * In addition, computes the two output bits:
 * if (out == 0) zr = 1, else zr = 0
 * if (out < 0) ng = 1, else ng = 0
 */
// Implementation: Manipulates the x and y inputs
// and operates on the resulting values, as follows:
// if (zx == 1) sets x = 0          // 16-bit constant
// if (nx == 1) sets x = !x        // bitwise not
// if (zy == 1) sets y = 0          // 16-bit constant
// if (ny == 1) sets y = !y        // bitwise not
// if (f == 1) sets out = x + y    // integer 2's complement addition
// if (f == 0) sets out = x & y    // bitwise and
// if (no == 1) sets out = !out    // bitwise not

CHIP ALU {
    IN
        x[16], y[16], // 16-bit inputs
        zx, // zero the x input?
        nx, // negate the x input?
        zy, // zero the y input?
        ny, // negate the y input?
        f, // compute (out = x + y) or (out = x & y)?
        no; // negate the out output?

    OUT
        out[16], // 16-bit output
        zr,      // if (out == 0) equals 1, else 0
        ng;      // if (out < 0) equals 1, else 0

    PARTS:
        // Step 1: Handle zx and nx (zero/negate x)
        Mux16(a=x, b=false, sel=zx, out=x1);
        Not16(in=x1, out=nx1);
        Mux16(a=x1, b=nx1, sel=nx, out=x2);

        // Step 2: Handle zy and ny (zero/negate y)
        Mux16(a=y, b=false, sel=zy, out=y1);
        Not16(in=y1, out=ny1);
        Mux16(a=y1, b=ny1, sel=ny, out=y2);

        // Step 3: f – AND or ADD
        And16(a=x2, b=y2, out=fAnd);
        Add16(a=x2, b=y2, out=fAdd);
        Mux16(a=fAnd, b=fAdd, sel=f, out=fOut);
```

```

// Step 4
Not16(in=f0Out, out=nf0Out);
Mux16(a=f0Out, b=nf0Out, sel=no, out=out, out[0..7]=low8, out[8..15]=high8, out[15]=ng);

// Step 5
Or8Way(in=low8, out=orLow);
Or8Way(in=high8, out=orHigh);
Or(a=orLow, b=orHigh, out=notZero);
Not(in=notZero, out=zr);
}

```

NAND2Tetris / Hardware Simulator / ALU.hdl

HDL --LBSt Project 2 ALU

Chip ALU Eval Reset Clock: 0

Input pins

x	0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1	de
y	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1	de
zx	0	
nx	1	
zy	0	
ny	1	
f	0	

Test Script Compare File Output File Diff Table

```

367 set zx 0,
368 set nx 1,
369 set zy 0,
370 set ny 1,
371 set f 0,
372 set no 1,
373 eval,
374 output;

```

Simulation successful: The output file is identical to the compare file

Simulator output: ALU